

A Radix-4 Booth Multiplier in Structural VHDL

Architecture, optimization and synthesis at 32 bits in Nangate 45 nm

Erfan Taheri

Nangate 45 nm Open Cell Library · Synopsys Design Compiler · ModelSim

Abstract

This report presents the design and optimization of a generic signed radix-4 Booth multiplier described in structural VHDL, together with the synthesis results obtained for the 32×32 -bit instance using Synopsys Design Compiler and the Nangate 45 nm library. Three reduction strategies are implemented behind a single entity interface: a behavioural summation of the partial products, a Wallace carry-save adder tree, and a Dadda tree. Three architectural optimizations are then applied in sequence: elimination of the partial-product sign extension in favour of a constant correction vector, replacement of the Wallace tree by a generic Dadda schedule computed at elaboration time, and fusion of the Booth selection and negation steps into a single compound gate per bit. Each optimization is derived analytically, implemented as an alternative VHDL architecture, verified in simulation and measured over a sweep of clock constraints. A multiplier inferred by the synthesis tool from a plain $A * B$ statement is synthesised under identical conditions and used as an absolute reference point.

The final design reaches a minimum achieved period of **1.76 ns** [R4] against **1.63 ns** [R6] for the inferred reference, and a minimum area of **5029 μm^2** [R4] against **4441 μm^2** [R6]. Relative to the unoptimized Wallace baseline, the three optimizations together recover **69%** of the area difference between the structural design and the inferred reference. Every measured quantity in this report carries a tag such as [R4] identifying the synthesis report it was read from; the key is given in Appendix B.

Contents

1	Introduction	2
2	Architecture	3
2.1	Radix-4 modified Booth recoding	3
2.2	Datapath organisation	3
2.3	Exactness modulo 2^{2N}	4
3	Optimization 1: elimination of the partial-product sign extension	4
3.1	Row format and the cost of explicit sign extension	4
3.2	The inversion identity	4
3.3	Closed form of the correction constant	5
3.4	Absorption into the Booth increment row	6
3.5	Implementation	6
3.6	Measured effect	6
4	Optimization 2: a generic Dadda reduction schedule	7
4.1	Column representation	7
4.2	Wallace and Dadda placement criteria	7
4.3	Generation of the initial array	8
4.4	The scheduling algorithm	8

4.5	Carry accounting	9
4.6	Slot allocation	9
4.7	Consistency and generic cost	10
4.8	Constraints imposed by VHDL-93	11
4.9	Measured effect	11
5	Optimization 3: fusion of the Booth selector	11
5.1	Diagnosis	11
5.2	Algebraic formulation	12
5.3	Completeness of the cover	12
5.4	Effect on technology mapping	12
5.5	Measured effect	12
6	Functional verification	13
7	Synthesis methodology	13
7.1	Flow and constraints	13
7.2	The achieved period metric	14
7.3	Non-monotonicity of the optimizer	14
8	Results	14
8.1	Area and delay	14
8.2	Cell-level analysis	15
8.3	Crossover in the mid-period region	16
9	Conclusions	16
A	Source organisation and variant selection	17
A.1	Directory layout	17
A.2	The packages	17
A.3	Variant selection	17
B	Provenance of the reported measurements	18

1 Introduction

A parallel multiplier is one of the few datapath blocks in which the choice of architecture, rather than the choice of library or the effort level of the synthesis tool, dominates the result. The objective of this work is to quantify that dependence: to implement a radix-4 Booth multiplier structurally, apply a sequence of well-defined architectural optimizations, and measure each one under an identical synthesis flow.

The design is generic in the operand width N . All results reported here are for $N = 32$, signed, with the product taken modulo 2^{2N} .

Three constraints govern the implementation throughout:

- **Single source tree.** Every variant is an alternative **architecture** of an existing entity, selected by a VHDL **configuration**. No file is duplicated, so a variant differs from its predecessor only in the component it binds.
- **Absolute reference.** A separate design consisting of the single statement $P \leq A * B$ is synthesised with the same script, constraints and library. It provides a fixed point against which the structural variants are measured, so that improvements are not reported relative to an arbitrary baseline alone.

- **Verification precedes measurement.** No variant is synthesised before it passes the self-checking testbench. This is not a formality: an early variant that skipped this step produced excellent area figures precisely because it was missing logic.

Section 2 describes the architecture and establishes the modular-arithmetic framework on which the first optimization depends. Sections 3–5 derive, implement and measure the three optimizations. Section 6 describes the verification strategy, Section 7 the synthesis methodology, and Section 8 the measured results. Source organisation and the configuration mechanism are documented in Appendix A.

2 Architecture

2.1 Radix-4 modified Booth recoding

A direct implementation of an $N \times N$ product accumulates one shifted copy of A per bit of B , giving N partial-product rows. Radix-4 modified Booth recoding [1, 2] inspects overlapping triplets of B — bits $(2i+1, 2i, 2i-1)$ with the convention $B_{-1} = 0$ — and maps each triplet to a single signed digit $d_i \in \{0, \pm 1, \pm 2\}$, so that

$$A \cdot B = \sum_{i=0}^{N/2-1} d_i A \cdot 4^i. \quad (1)$$

triplet	digit d_i	sel	row generated
000, 111	0	000	all zeros
001, 010	+1	001	$+A$
011	+2	011	$+2A$
100	-2	111	$-2A$
101, 110	-1	101	$-A$

Table 1: Radix-4 modified Booth recoding as implemented in `booth_encoder`. The control vector is `sel = {neg, double, enable}`.

The number of rows is therefore $N/2$ rather than N . This halving is the principal benefit of the recoding, because the reduction tree that follows is the dominant contributor to both area and delay, and its cost grows with the number of rows. Multiplication by two is a wired shift and costs nothing. Negation is not free: it is the origin of two of the three optimizations presented below.

2.2 Datapath organisation

The datapath is shown in Figure 1 and consists of five stages.

Stage	Entity	Function
encode	<code>booth_encoder</code>	triplet of $B \rightarrow \text{sel} = \{\text{neg}, \text{double}, \text{enable}\}$
generate	<code>mux_and_shift</code>	one partial-product row: $0, \pm A, \pm 2A$
correct	<code>corrector</code>	one additional row: Booth increments and the bias constant
reduce	<code>reduction_tree</code>	$N/2+1$ rows $\rightarrow$ carry and sum vectors
add	<code>P4_adder</code>	sparse-tree carry-propagate adder $\rightarrow 2N$ -bit product

Table 2: Stages of the structural multiplier. The synthesis top level, `boothmul_registered`, wraps this combinational block between input and output registers.

Row i carries weight 4^i , applied as a left shift of $2i$ positions in `boothmul`. The reduction tree is deliberately separated from the final carry-propagate adder, so that the reduction strategy can be replaced without touching any other file.

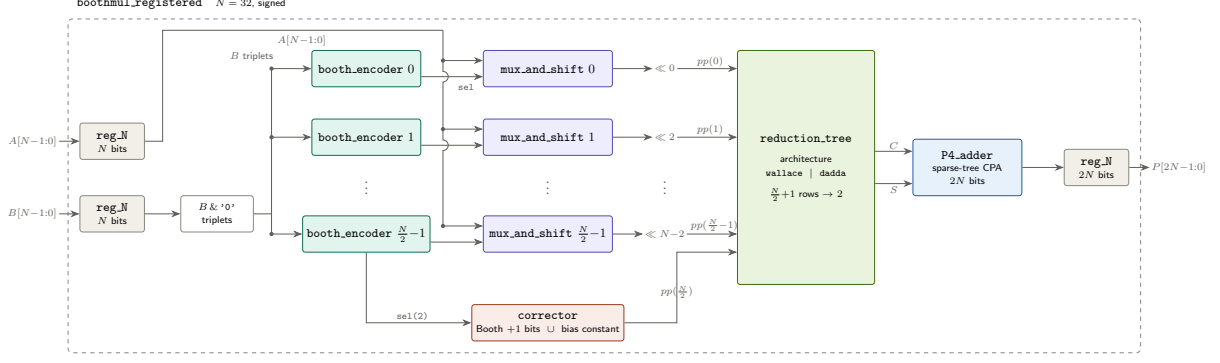


Figure 1: Top level of the structural multiplier. The partial-product rows are drawn generically: there are $N/2$ of them plus the corrector row. Only `reduction_tree` differs between the Wallace and Dadda variants, and only `mux_and_shift` differs between the sign-extension variants.

2.3 Exactness modulo 2^{2N}

Three mechanisms in this design discard information: the carry-save adders drop the carry generated by the most significant column, the final adder’s carry-out is left unconnected, and the sign-extension optimization of Section 3 deliberately introduces a bias that is cancelled later. All three are exact, for the same reason.

The datapath computes in the ring $\mathbb{Z}/2^{2N}\mathbb{Z}$. Every operation performed on the $2N$ -bit vectors — addition, carry-save addition, shifting, and the final carry-propagate addition — is a ring operation, so discarding any carry out of bit $2N - 1$ is a reduction modulo 2^{2N} and commutes with everything that follows. The product of two N -bit two’s complement operands lies in $[-2^{2N-2}, 2^{2N-2}]$, which is representable in $2N$ bits, so its residue class modulo 2^{2N} determines it uniquely.

It is therefore sufficient to show that the array of rows delivered to the reduction tree is congruent to $A \cdot B$ modulo 2^{2N} . Intermediate values may overflow freely; only the congruence matters. This property is what makes the derivation in Section 3 valid, and it is the reason a constant may be added to the array to cancel a bias without any range analysis.

3 Optimization 1: elimination of the partial-product sign extension

3.1 Row format and the cost of explicit sign extension

The term $m_i = d_i A$ generated for row i lies in $[-2^N, 2^N - 2]$ and is therefore representable in $N + 1$ bits of two’s complement. It must be placed in a $2N$ -bit array at weight 2^{2i} . Because m_i is signed and the array is summed as an unsigned quantity, bit N of the row must be replicated upward from position $N + 2i$ to position $2N - 1$.

The number of replicated bits in row i is $N - 1 - 2i$, so across the array

$$\sum_{i=0}^{N/2-1} (N - 1 - 2i) = \frac{N}{2}(N - 1) - \frac{N}{2}\left(\frac{N}{2} - 1\right) = \frac{N^2}{4} \quad (2)$$

partial-product bits exist only to carry sign information. At $N = 32$ this is 256 bits out of a total of 800, close to one third of the array. Each of them must be generated, routed, and compressed by the reduction tree, and none carries information beyond the single bit it replicates.

3.2 The inversion identity

Let s denote the sign bit of m_i , that is $m_{i,N}$. In two’s complement,

$$m_i = -s \cdot 2^N + \sum_{j=0}^{N-1} m_{i,j} 2^j. \quad (3)$$

Since $s + \bar{s} = 1$ for a single bit, $-s = \bar{s} - 1$, and therefore

$$-s \cdot 2^N = \bar{s} \cdot 2^N - 2^N. \quad (4)$$

Define the *stored* row as

$$R_i = \bar{s} \cdot 2^N + \sum_{j=0}^{N-1} m_{i,j} 2^j, \quad (5)$$

that is: the N low bits unchanged, the sign bit inverted, and zeros above. Substituting (4) gives the exact relation

$$m_i = R_i - 2^N. \quad (6)$$

The sign extension has been replaced by a single inverted bit. What remains is a constant error of exactly 2^N per row, independent of A , of the Booth digit, and of the row index. It is not an approximation, and it does not depend on the data.

3.3 Closed form of the correction constant

Applying (6) to (1), the array of stored rows overshoots the product by

$$E = \sum_{i=0}^{N/2-1} 2^{N+2i} = 2^N \cdot \frac{4^{N/2} - 1}{3} = \frac{2^N(2^N - 1)}{3}. \quad (7)$$

By Section 2.3 the correction is the additive inverse of E in $\mathbb{Z}/2^{2N}\mathbb{Z}$:

$$C = 2^{2N} - \frac{2^N(2^N - 1)}{3} = 2^N \left(2^N - \frac{2^N - 1}{3} \right). \quad (8)$$

Two properties of (8) are structurally important. First, C is a multiple of 2^N , so its lower N bits are zero. Second, for even N the quantity $(2^N - 1)/3$ has the binary form 0101...01, and subtracting it from 2^N yields 1010...1011. The constant therefore occupies bits $2N - 1$ down to N with the hexadecimal pattern AA...AB.

N	E	C , bits $2N-1$ down to N
4	80	0xB
8	21 760	0xAB
16	1.43×10^9	0xAAAAB
32	6.14×10^{18}	0xAAAAAAAB

Table 3: The correction constant for several operand widths. The repeating pattern is a direct consequence of the factor $1/3$ in (7).

The constant is generated at elaboration time by `sign_ext_const` in `common_pkg`. It is built bit by bit rather than arithmetically, because C is a $2N$ -bit quantity and the VHDL integer type cannot represent it at $N = 32$:

```
function sign_ext_const(NB : natural) return std_logic_vector is
    variable v : std_logic_vector(2*NB-1 downto 0) := (others => '0');
begin
    v(NB) := '1';
    -- the trailing B
```

```

for j in NB+1 to 2*NB-1 loop      -- the repeating A above it
  if (j mod 2) = 1 then v(j) := '1'; end if;
end loop;
return v;
end function;

```

3.4 Absorption into the Booth increment row

Radix-4 Booth already requires one additional row. A negated row is produced as a one's complement by `mux_and_shift`; the matching $+1$ is not added locally but collected across all rows, with `sel(2)` of row i placed at weight 2^{2i} . This increment row occupies only the even positions of the lower half of the array.

The correction constant C occupies only the upper half. The two therefore share one row without collision, and the entire cost of Optimization 1 is confined to the upper half of a row the design already had. The `corrector` entity builds it:

```

architecture no_sign_extend of corrector is
  constant SE_CONST : std_logic_vector(2*N-1 downto 0) := sign_ext_const(N);
begin
  gen_neg: for i in 0 to N/2-1 generate      -- low half: Booth increments
    correction_v(2*i)  <= neg_bits(i);
    correction_v(2*i+1) <= '0';
  end generate;
  correction_v(2*N-1 downto N) <= SE_CONST(2*N-1 downto N); -- high half
end architecture;

```

The high half is a tie to the supply rails. It contains no logic, and the synthesis tool propagates it as constants through the first reduction stage.

3.5 Implementation

The two row formats are alternative architectures of `mux_and_shift`. The baseline sign-extends to $2N$ bits; the optimized version constructs the row in $N + 1$ bits, inverts bit N , and zero-fills above:

```

-- architecture sign_extend
q := std_logic_vector(resize(signed(A), 2*N));      -- full-width row
...
q := q xor (q'range => sel(2));
pp <= q;

-- architecture no_sign_extend
q := std_logic_vector(resize(signed(A), N+1));      -- N+1 bits only
...
q := q xor (q'range => sel(2));
q(N) := not q(N);                                  -- stored sign
pp <= std_logic_vector(resize(unsigned(q), 2*N));

```

The `mux_and_shift` and `corrector` architectures must be bound in matching pairs. The `no_sign_extend` row format biases every row by $+2^{2i+N}$ and depends on the matching corrector architecture to cancel it. The configurations in `cfg/` enforce the pairing; the language does not.

3.6 Measured effect

The array shrinks from 800 to 561 partial-product bits, a reduction of 29.9%. At relaxed timing the synthesised area falls from $6346 \mu\text{m}^2$ [R1] to $5975 \mu\text{m}^2$ [R2], a reduction of 5.8%, and the cell count at a 3.0 ns constraint falls from 4639 to 4455 [R7].

The distance between a 29.9% reduction in partial-product bits and a 5.8% reduction in area is itself informative. The removed bits are constant-valued sign replications, and constant

propagation in `compile_ultra` already eliminates much of the logic that would otherwise process them. What the optimization removes with certainty, and what constant propagation cannot remove, is the structural height of the columns. That is what Optimization 2 exploits.

4 Optimization 2: a generic Dadda reduction schedule

The reduction tree is the largest block in the design. This section describes its replacement by a Dadda schedule computed entirely at elaboration time, for arbitrary N , from the geometry of the partial-product array.

4.1 Column representation

The partial-product array is a set of $2N$ columns; column k contains n_k bits, referred to below as *dots*, all of weight 2^k . A reduction network compresses every column until no column contains more than two dots, at which point a carry-propagate adder completes the sum.

The two primitives are the full adder, which takes three dots of column k and produces one dot in column k (sum) and one in column $k + 1$ (carry), and the half adder, which takes two and produces the same two outputs. A full adder reduces the total dot count by one; a half adder does not reduce it at all.

This gives an exact invariant, used as a correctness check in Section 4.7: if the array begins with D_0 dots and terminates with D_f , the number of full adders required is exactly $D_0 - D_f$, independently of the schedule. Half adders fall outside this accounting and are therefore worth minimising.

4.2 Wallace and Dadda placement criteria

Wallace reduction [3] compresses as early as possible: at every stage, every group of three dots in a column is given a full adder. Dadda reduction [4] compresses as late as possible: a target height is fixed for each stage, and only enough compressors are placed to meet it.

The targets follow the Dadda sequence

$$d_0 = 2, \quad d_{j+1} = \left\lfloor \frac{3}{2} d_j \right\rfloor \implies 2, 3, 4, 6, 9, 13, 19, 28, \dots \quad (9)$$

where d_j is the largest column height reducible to two in j stages. The number of stages needed for r rows is the smallest j with $d_j \geq r$. For $N = 32$ the array has $N/2 + 1 = 17$ rows, so $d_6 = 19 \geq 17$ and six reduction stages are required, with targets 13, 9, 6, 4, 3, 2.

Both quantities are implemented in `dadda_types_pkg`:

```
function height_dadda(lev : natural) return natural is
  variable temp : integer := 2;
begin
  for i in 0 to lev-1 loop temp := (temp*3)/2; end loop;
  return temp;
end function;

function num_layers_dadda(NROWS : natural) return natural is
  variable temp : natural := 2; variable level : natural := 1;
begin
  while NROWS > temp loop
    temp := height_dadda(level);
    level := level + 1;
  end loop;
  return level;
end function;
```

NUM_LAYERS_D is declared as a deferred constant in the package header and given its value in the package body, so that the header can expose it to the type declarations before `num_layers_dadda` is defined.

The practical difference between the two strategies is the half-adder count. A Wallace tree places a half adder wherever a column has two spare dots, and each one costs cells without reducing the dot count. Dadda places a half adder only when a column is exactly one dot above its target. For $N = 32$ the schedule computed here uses 45 half adders across the entire array.

4.3 Generation of the initial array

The schedule is derived from the geometry of the array, not from its values, so it can be computed before any signal exists. The function `making_initail_layer` returns a Boolean occupancy matrix for the sign-extension-eliminated layout of Section 3:

```
-- rows 0 .. N/2-1 : N+1 significant bits, shifted by 2i
loop_rows: for ro in 0 to (NBIT/2-1) loop
  for col in ro*2 to ro*2+NBIT loop
    rows(ro)(col) := '1';
  end loop;
end loop;

-- row N/2 : the corrector row
for i in 0 to ((NBIT-1)/2) loop
  rows(last_row)(i*2) := '1';
end loop;
rows(last_row)(NBIT) := '1';
for i in 0 to ((NBIT-1)/2) loop
  rows(last_row)(i*2 + NBIT+1) := '1';
end loop;
```

The three parts of the corrector row correspond term by term to the decomposition of Section 3: the Booth increments in the low half, the bit at position N , and the alternating pattern above it. Because this matrix hard-codes the optimized layout, the Dadda architecture is valid only in combination with `mux_and_shift(no_sign_extend)` or `mux_and_shift(fused_selector)`; no configuration pairs it with the baseline.

4.4 The scheduling algorithm

`constructing_dadda` performs the entire schedule at elaboration time and returns three matrices of identical shape, indexed by (layer, row slot, column):

- `FA_matrix` — marks the three source slots of every full adder;
- `HA_matrix` — marks the two source slots of every half adder;
- `remaining_system` — the dots no compressor consumed, which pass unchanged to the next layer.

The three are produced in a single pass. A mark-and-clear discipline guarantees that they stay mutually consistent: when a dot is assigned to a compressor its position is written into the corresponding matrix and simultaneously cleared from the working system, so it can neither be assigned twice nor survive into the set of passed dots.

```
loop_setting_FAs: for i in 0 to (fa_id-1) loop
  node_cnt := 0; roww := 0;
  while node_cnt < 3 loop
    if system(lev)(roww)(col) = '1' then
      node_cnt := node_cnt + 1;
      FA_matrix(lev)(roww)(col) := '1'; -- record the source slot
    end if;
    roww := roww + 1;
  end loop;
end loop;
```

```

    system(lev)(roww)(col)    := '0';    -- and consume it
  end if;
  roww := roww + 1;
end loop;
end loop;

```

Layers are processed from the initial layer downward, and columns within a layer from least to most significant, because the compressor count of a column depends on the carries produced by the column below it.

4.5 Carry accounting

The height test is the subtle part of the algorithm. The target height applies to layer $\text{lev} - 1$, and column k of that layer receives dots from three distinct sources:

1. C_{in} carries generated by column $k - 1$ of the *same* stage, written into layer $\text{lev} - 1$ when column $k - 1$ was processed;
2. one sum bit per compressor placed in column k ;
3. the dots of column k that no compressor consumed.

If column k holds n dots at layer lev and is given F full adders and H half adders, the resulting height at layer $\text{lev} - 1$ is

$$C_{\text{in}} + (F + H) + (n - 3F - 2H) = C_{\text{in}} + n - 2F - H, \quad (10)$$

so the compressor count must satisfy $C_{\text{in}} + n - 2F - H \leq d_{\text{lev}-1}$. A column must therefore be sized against a quantity that does not exist in its own layer. Omitting C_{in} from the test yields a schedule that is locally plausible and globally wrong: columns overflow their target height and the array fails to reach two rows in the allotted number of stages.

The loop implements exactly this, using the fact that a full adder removes two dots from the height and a half adder removes one:

```

count_temp := count + C_out_count;    -- own dots plus carries from col-1
while (count_temp > max_height) loop
  if count_temp >= (max_height + 2) then
    count_temp := count_temp - 2; fa_id := fa_id + 1;
  elsif count_temp = (max_height + 1) then
    ha_flag := 1; count_temp := count_temp - 1;
  end if;
end loop;

```

The half-adder branch is reached only when the residual excess is exactly one, which is the property that keeps the half-adder count low.

4.6 Slot allocation

A compressor instantiated in the netlist must know which row slot of the next layer its outputs drive. The allocation order for column k of layer $\text{lev} - 1$ is fixed:

slots $0 \dots C_{\text{in}} - 1$	carries arriving from column $k - 1$
slots $C_{\text{in}} \dots C_{\text{in}} + F + H - 1$	sums of this column's compressors
remaining slots	dots passed through unchanged

This order must be reproduced identically in two places: in `constructing_dadda`, which writes the matrices, and in `dadda_tree`, which writes the port maps. The difficulty is that a `generate` statement carries no sequential state — each instance is elaborated independently and cannot

Function	Returns
<code>num_rows_dadda(lev)</code>	occupied row slots at layer <code>lev</code>
<code>fa_count_col_dadda(...)</code>	number of full adders in a column
<code>ha_flag_col_dadda(...)</code>	1 if the column has a half adder, else 0
<code>num_C_out_from_prev_col(...)</code>	C_{in} , recomputed from column $k - 1$
<code>fa_arg_func(...)</code>	the three source row indices of each full adder
<code>ha_arg_func(...)</code>	the two source row indices of the half adder
<code>remaining_counting_func(...)</code>	number of passed dots in a column
<code>remaining_bit_pos(...)</code>	their row indices

Table 4: Analysis functions used to recover port-map positions from the scheduling matrices. All are pure and evaluated at elaboration time.

inherit a running counter from the previous column — so every offset must be recomputable from the matrices alone. This is the purpose of the analysis functions in `dadda_math_pkg`.

`num_C_out_from_prev_col` resolves the generate-statement problem directly: instead of propagating C_{in} forward, it recomputes it by counting the compressors of column $k - 1$ in the same layer. Two assertions guard the invariants at elaboration time — that no column is assigned more than one half adder, and that the half-adder source positions are unique. A violation fails elaboration rather than producing a silently mis-wired tree.

The structural instantiation then reads directly from the matrices. The generate nest is three levels deep — layer, column, compressor — and its constant declarations carry all the positional information:

```
columns: for col in 0 to NBIT*2-1 generate
  constant fa_count_col    : integer := fa_count_col_dadda(FA_matrix, lev, num_rows_current, col);
  constant C_out_counting  : integer := num_C_out_from_prev_col(FA_matrix, HA_matrix, lev, col);
  constant FA_args        : fa_args_t(0 to fa_count_col-1)
                          := fa_arg_func(FA_matrix, num_rows_current, fa_count_col, lev, col);
begin
  mapping_fa: for ID in 0 to fa_count_col-1 generate
    constant sum_bit_arg    : integer := ID + C_out_counting;
    constant carry_out_arg  : integer := ID;
    begin
      FA_i : FA port map (
        A => layer(lev)(FA_args(ID)(0))(col),
        B => layer(lev)(FA_args(ID)(1))(col),
        Ci => layer(lev)(FA_args(ID)(2))(col),
        S => layer(lev-1)(sum_bit_arg)(col),
        Co => layer(lev-1)(carry_out_arg)(col+1));
    end generate;
end generate;
```

4.7 Consistency and generic cost

The schedule was validated against the dot-count invariant. For the optimized layout at $N = 32$ the array begins with $D_0 = 561$ dots and terminates with $D_f = 126$, so exactly $561 - 126 = 435$ full adders are necessary; the schedule produces 435. The identity holds at every width tested, and row 2 of the final layer is empty at all widths, confirming that the reduction really does terminate at two rows.

Table 5 quantifies the structural difference. The Wallace implementation instantiates carry-save adders across the full $2N$ width, giving 960 full adder instances at $N = 32$ against 435 full adders and 45 half adders for Dadda. Many of the Wallace instances have constant inputs and are removed by the synthesis tool, so the difference in the synthesised result is smaller than the ratio suggests. It is not zero, because constant propagation cannot recover the shorter critical path that a column-scheduled tree provides.

N	rows	array dots		Dadda				Wallace
		baseline	optimized	stages	FA	HA	cells	FA instances
8	5	56	45	3	15	9	24	48
16	9	208	153	4	91	21	112	224
32	17	800	561	6	435	45	480	960
64	33	3136	2145	8	1891	93	1984	3968

Table 5: Cost of the reduction network as a function of operand width, computed from the elaboration-time schedule. The Wallace column gives the number of full adder instances written by the row-wise implementation: $2N$ per carry-save adder, with no allowance for columns that are not full.

4.8 Constraints imposed by VHDL-93

Three language restrictions shaped the implementation.

Generate bounds must be globally static. The number of compressors in a column is data of the schedule, so the schedule must be available as a constant expression before elaboration of the generate statement. This is why it is computed by a pure function over Boolean matrices rather than derived incrementally while the structure is built.

A package cannot see a generic. The port type `pp_array` must be declared in a package so that both the entity and its instantiators can see it, and a VHDL-93 package has no visibility of generics. The operand width is therefore a constant, `NBIT` in `common_pkg`, rather than a generic of the top-level entity. The design remains generic in the sense that every downstream quantity is a function of `NBIT`; changing that one constant reconfigures the multiplier, the reduction schedule and the testbench together.

Deferred constants. `NUM_LAYERS_D` is required by the type declarations in the package header but computed by a function defined in the package body. It is declared without a value in the header and given one in the body.

4.9 Measured effect

At relaxed timing the area falls from $5975\text{ }\mu\text{m}^2$ [R2] to $5620\text{ }\mu\text{m}^2$ [R3], a further 5.9%, and the minimum achieved period improves from 1.87 ns to 1.78 ns [R3] — the largest single improvement in delay among the three optimizations. The cell count at a 3.0 ns constraint falls from 4455 to 3936 [R7].

5 Optimization 3: fusion of the Booth selector

5.1 Diagnosis

After Optimization 2 the design remained $1180\text{ }\mu\text{m}^2$ larger than the inferred reference. A cell census of the two netlists at a 3.0 ns constraint [R7] localised the difference.

	XOR + XNOR	FA cells	INV	total cells
Structural, Dadda	1230	117	505	3936
Inferred $A * B$	559	268	311	3035

Table 6: Cell census at a 3.0 ns constraint [R7], before Optimization 3.

The structural design contained 671 more exclusive-or cells than the inferred one, and mapped only 117 of its 435 scheduled full adders to library adder cells. The two observations have a common cause. `mux_and_shift` builds a row in two steps: a multiplexer selects A , $2A$ or zero, and every bit of the result is then exclusive-ored with `sel(2)` to perform the negation. That is $N + 1$ XOR2 cells per row and 16 rows, or 528 cells at $N = 32$, approximately $843 \text{ } \mu\text{m}^2$ performing nothing but conditional inversion.

The secondary effect is more damaging than the direct cost. A full adder is itself an exclusive-or-dominated function. With a layer of XOR gates immediately feeding the reduction tree, the logic cones presented to the technology mapper mix negation and compression, and the mapper decomposes them into generic gates instead of recognising the `FA_X1` pattern. The low adder-cell count is a symptom of the XOR layer, not an independent problem.

5.2 Algebraic formulation

The multiplexer and the conditional inversion can be merged into one function. Let

$$\text{src}_1 = A_{N-1} \parallel A \quad (A \text{ in } N + 1 \text{ bits}) \quad (11)$$

$$\text{src}_2 = A \parallel '0' \quad (2A \text{ in } N + 1 \text{ bits}) \quad (12)$$

and decode the control vector once per row into four mutually exclusive conditions:

$$\begin{aligned} c_{+1} &= \text{sel}(0) \cdot \overline{\text{sel}(1)} \cdot \overline{\text{sel}(2)}, & c_{-1} &= \text{sel}(0) \cdot \overline{\text{sel}(1)} \cdot \text{sel}(2), \\ c_{+2} &= \text{sel}(0) \cdot \text{sel}(1) \cdot \overline{\text{sel}(2)}, & c_{-2} &= \text{sel}(0) \cdot \text{sel}(1) \cdot \text{sel}(2). \end{aligned} \quad (13)$$

Each output bit is then a four-term sum of products:

$$q_i = c_{+1} \text{src}_{1,i} + c_{-1} \overline{\text{src}_{1,i}} + c_{+2} \text{src}_{2,i} + c_{-2} \overline{\text{src}_{2,i}}, \quad (14)$$

after which sign-extension elimination is applied exactly as before: $pp_N = \overline{q_N}$, $pp_{N-1:0} = q_{N-1:0}$, and zeros above.

5.3 Completeness of the cover

Equation (14) enumerates four of the five possible row values; the zero row is covered implicitly, because all four conditions are false when `sel(0)` = 0 and the sum of products evaluates to zero. The cover is complete provided the encoder never asserts `sel(2)` with `sel(0)` deasserted — that is, never requests a negated zero row. Table 1 confirms this: the only codes with `sel(2)` = 1 are 101 and 111, both of which have `sel(0)` = 1. The property follows from the encoding table and is not an assumption imposed on it.

5.4 Effect on technology mapping

Equation (14) is a two-level sum of products over four literals, and the Nangate library provides compound AND-OR-invert cells that implement it in a single stage. No exclusive-or appears anywhere in the row generator. The complemented terms $\overline{\text{src}_1}$ and $\overline{\text{src}_2}$ are the same function of A in all $N/2$ rows, so common-subexpression elimination shares one set of inverters across the array.

Exclusive-or cells fall from 1230 to 85, and the adder-cell count rises from 117 to 448, within 7% of the 480 compressors the schedule contains. Removing the XOR layer allowed the mapper to recognise compressors that had been present in the RTL all along, which is why this optimization produces the largest area gain of the three despite consisting of a rewrite of a single architecture.

5.5 Measured effect

Area falls from $5620 \text{ } \mu\text{m}^2$ [R3] to $5029 \text{ } \mu\text{m}^2$ [R4], a reduction of 10.5%, and the minimum achieved period improves from 1.78 ns to 1.76 ns [R4]. Total cell count falls by 28%.

	XOR + XNOR	FA cells	AOI/OAI	total cells	area
Dadda, separate negation	1230	117	1405	3936	5725 μm^2
Dadda, fused selector	85	448	1311	2819	5125 μm^2
Inferred $A * B$	559	268	950	3035	4910 μm^2

Table 7: Cell census at a 3.0 ns constraint [R7]. The adder-cell count rises from 117 to 448, close to the 435 full adders and 45 half adders the schedule specifies.

6 Functional verification

All variants share one self-checking testbench, `tb_multiplier`, which reads the operand width from `common_pkg` and selects its coverage mode accordingly.

NBIT	coverage
≤ 8	exhaustive — all 2^{2N} operand pairs (65 536 at $N = 8$)
> 8	corner cases $\{0, \pm 1, 2^{N-1} - 1, -2^{N-1}\}$ plus 20 000 random pairs

The reference model is `signed(A) * signed(B)` truncated to $2N$ bits, evaluated independently of the design. Mismatches are counted and reported, so a failing variant produces a mismatch count rather than a single assertion.

Every variant is verified exhaustively at $N = 8$ before being synthesised at $N = 32$. Exhaustive coverage at the smaller width is more valuable than random coverage at the larger one, because the properties under test — the recoding table, the bias cancellation, the completeness of the fused selector’s cover and the column scheduling — are structural, and their failure modes appear at $N = 8$ exactly as they would at $N = 32$.

Two defects were found this way that would not have been found otherwise. In the first, the $-2A$ row was formed by shifting without complementing, failing 256 of 1280 directed cases. In the second, a behavioural variant omitted the Booth increment entirely and failed 61 440 of 65 536 exhaustive cases; its synthesis results were excellent because the missing logic was genuinely absent. Both defects were invisible in the area and timing reports and immediately visible in simulation.

Each synthesis configuration has a matching testbench configuration (Appendix A), so the binding verified in simulation is the binding that is synthesised.

7 Synthesis methodology

7.1 Flow and constraints

The synthesis top level is `boothmul_registered`, which places the combinational multiplier between input and output registers. This makes the critical path internal — register to register through the multiplier — so that the reported timing measures the design rather than the test environment.

The input and output delays are absolute values rather than fractions of the clock period. This is deliberate and is a precondition for the metric defined in Section 7.2: if the port constraints scaled with the period, the slack reported at one period could not be compared with the slack reported at another. Because the design is registered on both sides, the port paths terminate at a flip-flop within a few gates and never become critical, even at the tightest constraint. The timing reports confirm this: the critical path runs from a bit of the B register to the most significant bit of the product register in every run of the sweep.

The clock and reset networks are marked `set_dont_touch_network` and `set_ideal_network`, since clock tree construction belongs to place-and-route and not to this flow.

Library	NangateOpenCellLibrary, typical ECSM
Tool	Synopsys Design Compiler W-2024.09-SP2, <code>compile_ultra</code>
Wire load model	5K_hvratio_1_4
Clock	<code>create_clock -period \$clockPeriod</code> , swept
Clock uncertainty	0.05 ns; transition 0.05 ns; latency 0.05 ns
Input delay	0.25 ns, absolute
Output delay	0.15 ns, absolute
Driving cell	BUF_X4; output load 0.05 pF
Max transition	0.20 ns
Area constraint	<code>set_max_area 0</code>
Sweep	21 periods from 1.0 to 6.0 ns, 6 configurations

Table 8: Synthesis environment. The same constraint file is sourced once per period; `$clockPeriod` is the only quantity that changes.

7.2 The achieved period metric

A synthesis run that fails to meet its constraint still produces a valid netlist; it is simply slower than requested. Discarding such runs would discard the fastest netlists in the sweep, because the tool produces its most aggressive structures only when the constraint is tight enough to force them.

The quantity reported here is therefore the period the netlist actually supports:

$$T_{\text{achieved}} = T_{\text{constraint}} - \text{slack}. \quad (15)$$

Slack is negative for a violated path, so a violated run yields $T_{\text{achieved}} > T_{\text{constraint}}$. The identity is exact only when every timing quantity other than the clock is independent of $T_{\text{constraint}}$, which is why the port delays in Table 8 are absolute; clock uncertainty, transition and latency are likewise fixed.

As an example, the fused-selector variant constrained at 1.0 ns reports a slack of -0.76 ns [R4], so the netlist supports 1.76 ns. No run in the sweep produced a faster netlist for that variant, and the tightest constraint gave the fastest netlist for every variant, which indicates that the left edge of the achievable region has not been reached at 1.0 ns.

7.3 Non-monotonicity of the optimizer

Area is not a monotonic function of the clock constraint under `compile_ultra`. Relaxing the period can produce a larger netlist: for the Wallace baseline the run at 1.1 ns is larger than the run at 1.0 ns [R1], and similar reversals occur elsewhere in the sweep.

This was verified not to be an artefact of the area constraint. A controlled sweep with `set_max_area 0` removed [R8] gave identical minimum achieved periods and area differences below 3%, and remained non-monotonic. The cause is the heuristic nature of the optimizer, not the constraint.

All curves in Section 8 are therefore Pareto fronts computed over the whole sweep rather than raw per-constraint traces: a point is plotted only if no other run of the same variant is both faster and smaller. Dominated points are omitted.

8 Results

8.1 Area and delay

Table 9 gives the extremes of each variant. The three optimizations together recover 69.1% of the area difference between the structural baseline and the inferred reference, and reduce the delay difference from 0.23 ns to 0.13 ns.

Variant	min. achieved	min. area	gap closed	ref.
Wallace, sign-extended (baseline)	1.86 ns	6346 μm^2	—	[R1]
+ sign-extension elimination	1.87 ns	5975 μm^2	19.5%	[R2]
+ Dadda reduction	1.78 ns	5620 μm^2	38.1%	[R3]
+ fused selector	1.76 ns	5029 μm^2	69.1%	[R4]
Behavioural Booth (adder chain)	2.01 ns	6210 μm^2	7.1%	[R5]
Inferred A * B	1.63 ns	4441 μm^2	reference	[R6]

Table 9: Summary of the sweep. Periods are achieved values per (15); areas are the minimum over all constraints. *Gap closed* is the fraction of the area difference between the baseline and the inferred reference that the variant recovers.

The behavioural Booth variant, which replaces the reduction tree with a chain of signed additions, is the slowest design in the sweep at 2.01 ns [R5]. It is nevertheless smaller than the Wallace baseline at relaxed timing, because the synthesis tool implements the adder chain with its own carry-save structures rather than the explicitly instantiated ones. It is included as a measurement of what inference achieves when given a structurally poor description, and its timing result is the more informative of its two figures.

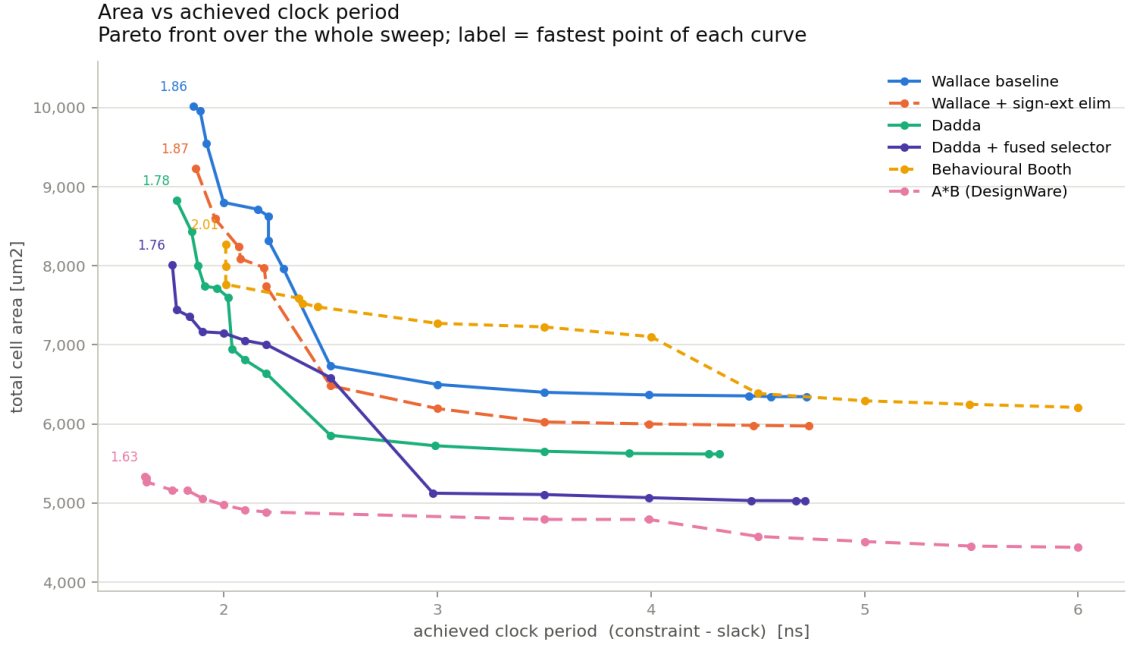


Figure 2: Area against achieved period. Each curve is the Pareto front of one variant over the whole 21-point sweep; dominated runs are omitted. Lower and further left is better.

Figure 2 shows the full fronts. The ordering of the three optimizations is consistent across the range of periods: each variant dominates its predecessor almost everywhere, and the separation is largest at relaxed timing, where the optimizer is free to minimise area rather than spending cells on speed.

8.2 Cell-level analysis

Table 7 gave the cell census at a fixed 3.0 ns constraint. Two observations are worth restating.

First, the area reduction from Optimization 1 (5.8%) is much smaller than the reduction in partial-product bits (29.9%). Constant propagation removes most of the logic associated with sign replication before the optimization is applied, so what the optimization contributes is the

reduction in structural column height, which is realised by Optimization 2.

Second, the three optimizations are not independent. Optimization 3 could not have produced its measured gain without Optimization 2 in place, because the compressors that the mapper was then able to recognise are the ones the Dadda schedule instantiates. Measured in isolation, each optimization would appear smaller than it does in sequence.

8.3 Crossover in the mid-period region

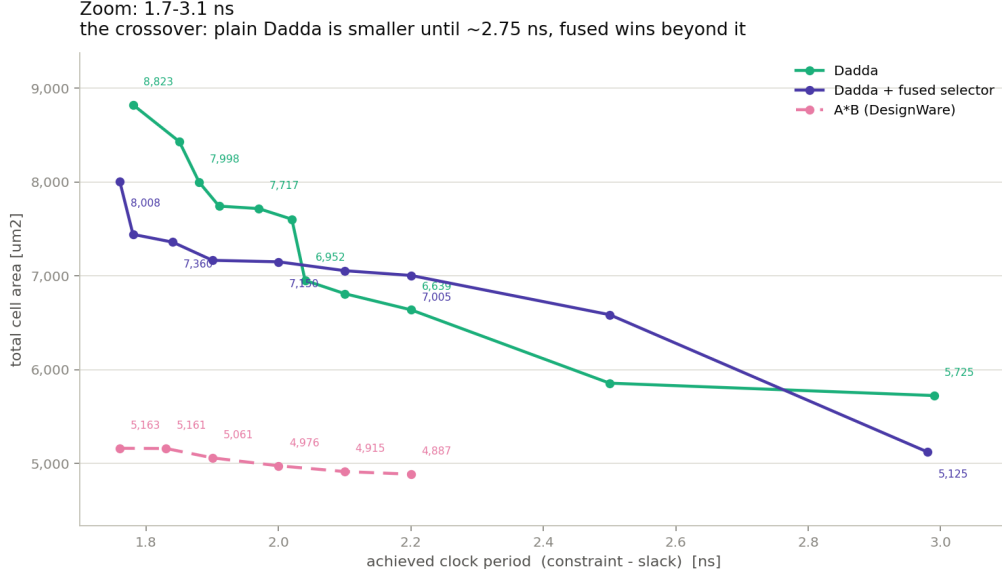


Figure 3: Detail of the region between 1.7 and 3.1 ns. The two Dadda variants cross twice; near 2.0 ns the variant with separate negation is the smaller of the two.

The two Dadda variants are not ordered uniformly. Figure 3 shows that they cross twice, and that near 2.0 ns the variant with separate negation is the smaller of the two. The fused selector presents deeper compound gates to the optimizer; at intermediate constraints the tool must upsize them to meet timing, and the upsizing costs more than the exclusive-or cells that were removed. At relaxed constraints the upsizing is unnecessary and the fused variant is smaller by 591 μm^2 ; at tight constraints it is faster.

This is an architectural trade-off rather than optimizer noise: the crossing is reproduced across adjacent constraint points, and it appears in the cell census as a shift in drive strengths rather than in cell counts.

9 Conclusions

A signed 32×32 radix-4 Booth multiplier was implemented in structural VHDL and optimized in three stages. Measured against an unoptimized Wallace baseline and an inferred $A * B$ reference synthesised under identical conditions, the result is a minimum achieved period of 1.76 ns against 1.63 ns for the reference, and a minimum area of 5029 μm^2 against 4441 μm^2 , recovering 69.1% of the initial area difference.

The individual contributions were unequal in a way not predictable from the analysis alone. Sign-extension elimination removes 29.9% of the partial-product bits but only 5.8% of the area, because constant propagation had already removed most of the associated logic. The Dadda schedule contributes a comparable area reduction and the largest single improvement in delay. Fusing the Booth selector contributes the largest area reduction of the three, and does so indirectly:

its measurable effect is not the 528 exclusive-or cells it removes but the 331 additional full adders the technology mapper is then able to recognise.

The general observation is that the cost of an RTL construct is determined as much by what it prevents the synthesis tool from recognising as by the gates it directly implies. In this design a single layer of exclusive-or gates between the row generators and the reduction tree was worth approximately 600 μm^2 — not through its own area, but by concealing the structure of everything behind it.

Four directions remain open: switching-activity-based power analysis, for which the removal of 1145 exclusive-or cells should be more visible than it is in area; gate-level simulation of each netlist against its SDF; extension of the sweep below 1.0 ns, since every variant reached its fastest netlist at the tightest constraint attempted; and a Baugh–Wooley [5] implementation as an alternative treatment of the sign problem.

A Source organisation and variant selection

A.1 Directory layout

Path	Contents
rtl/common/	iv, nd2, mux21, mux21_generic, fa, ha
rtl/adder/	the P4 sparse-tree carry-propagate adder chain
rtl/multiplier/	encoder, row generator, corrector, CSA, both trees, top levels
rtl/multiplier/packages/	the four elaboration-time packages
tb/	self-checking RTL and gate-level testbenches
cfg/	design, testbench and synthesis configurations
sim/	compile.do, sim.do
synthesis/syn/	synthesis.tcl, the SDC, and the report tree
synthesis/plot_pareto.py	reads the reports and produces the figures and tables

A.2 The packages

Package	Contents
common_pkg	NBIT, NROWS, pp_word, pp_array, sign_ext_const
wallace_math_pkg	row counts and layer depth for the uniform carry-save tree
dadda_types_pkg	the Dadda height sequence, the matrix types, NUM_LAYERS_D
dadda_math_pkg	the scheduling function and the port-map analysis functions

NBIT is the single width parameter of the project. The design, the reduction schedule and the testbench all derive from it, so one edit reconfigures all three consistently.

A.3 Variant selection

Each variant is a distinct architecture bound by a configuration. Instances inside a **generate** statement require a nested block configuration:

```
configuration CFG_BOOTHMUL_DADDA_FUSED_SEL of BOOTHMUL is
  for STRUCTURAL
    for gen_stages
      for mux_i : mux_and_shift
        use entity work.mux_and_shift(fused_selector);
      end for;
    end for;
    for corr_i : corrector
```

```

    use entity work.corrector(no_sign_extend);
end for;
for tree_i : REDUCTION_TREE
    use entity work.REDUCTION_TREE(dadda);
end for;
end for;
end configuration;

```

A configuration of BOOTHMUL has no effect on its own, because the testbench instantiates a *component*. Without a second, testbench-level configuration the default binding rule silently selects the most recently analysed architecture. Each design configuration therefore has a matching `cfg_tb_*` in `cfg/configurations_tb.vhd` and a matching `CFG_BOOTHMUL_REG_*` in `cfg/configurations_synthesis.vhd`, so that the binding simulated is the binding synthesised.

Binding	Effect
<code>mux_and_shift(sign_extend) + corrector(sign_extend)</code>	baseline
<code>mux_and_shift(no_sign_extend) + corrector(no_sign_extend)</code>	Optimization 1
<code>mux_and_shift(fused_selector) + corrector(no_sign_extend)</code>	Optimizations 1 and 3
<code>reduction_tree(wallace) / reduction_tree(dadda)</code>	Optimization 2
<code>mux_and_shift(behavioural)</code>	adder chain
<code>super_beh_multiplier_registered</code>	inferred $A * B$

B Provenance of the reported measurements

All measurements are read from the reports in `synthesis/syn/reports/`, one directory per configuration and one file per clock period. File names follow the pattern

`<report>_<CONFIG>_<period>.rpt`

where the period is written with `p` in place of the decimal point.

Tag	Configuration directory	Variant
[R1]	<code>CFG_BOOTHMUL_REG_WAL_BASE</code>	Wallace, sign-extended
[R2]	<code>CFG_BOOTHMUL_REG_WAL_OPT</code>	Wallace, sign extension eliminated
[R3]	<code>CFG_BOOTHMUL_REG_DADDA</code>	Dadda, separate negation
[R4]	<code>CFG_BOOTHMUL_REG_DADDA_FUSED_SEL</code>	Dadda, fused selector
[R5]	<code>CFG_BOOTHMUL_REG_BEH</code>	behavioural Booth, adder chain
[R6]	<code>CFG_REG_SUPER_BEH</code>	inferred $A * B$
[R7]	all of the above, period 3p0	cell census
[R8]	<code>reports_no_area_constraint/</code>	controlled run without <code>set_max_area</code>

Quantity	File	Field
Area	<code>area_*.rpt</code>	Total cell area
Cell counts	<code>reference_*.rpt</code>	Count column, summed by cell family
Slack	<code>timing_*.rpt</code>	slack (MET VIOLATED)
Critical path ends	<code>timing_*.rpt</code>	Startpoint, Endpoint
Achieved period	derived	(15)

Structural quantities — partial-product bit counts, compressor counts and the stage schedule — are not read from reports. They are computed from the elaboration-time functions in `dadda_math_pkg` and `wallace_math_pkg`, and were cross-checked against the dot-count invariant of Section 4.7.

The figures and the summary table are regenerated by

```
cd synthesis
python3 plot_pareto.py syn/reports -o ../docs/figures --zoom 1.7 3.1
```

which parses the report tree directly, with no intermediate data file.

References

- [1] A. D. Booth, “A Signed Binary Multiplication Technique,” *Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.
- [2] O. L. MacSorley, “High-Speed Arithmetic in Binary Computers,” *Proceedings of the IRE*, vol. 49, no. 1, pp. 67–91, 1961.
- [3] C. S. Wallace, “A Suggestion for a Fast Multiplier,” *IEEE Transactions on Electronic Computers*, vol. EC-13, no. 1, pp. 14–17, 1964.
- [4] L. Dadda, “Some Schemes for Parallel Multipliers,” *Alta Frequenza*, vol. 34, pp. 349–356, 1965.
- [5] C. R. Baugh and B. A. Wooley, “A Two’s Complement Parallel Array Multiplication Algorithm,” *IEEE Transactions on Computers*, vol. C-22, no. 12, pp. 1045–1047, 1973.
- [6] *Nangate 45 nm Open Cell Library*, Nangate Inc.
- [7] *Design Compiler User Guide*, W-2024.09-SP2, Synopsys Inc.